



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Estudio e implementación paralela de métodos semiimplícitos multipaso

Resolución de Ecuaciones de Advección-Reacción-Difusión

Autor

José Manuel Navarro Cuartero

Director

José Miguel Mantas Ruiz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

**Estudio e implementación paralela de métodos
semiimplícitos multipaso:
Resolución de Ecuaciones de Advección-Reacción-Difusión**

José Manuel Navarro Cuartero

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

First name, Family name (student)

Keywords: Keyword1, Keyword2, Keyword3,

Abstract

Write here the abstract in English.

Yo, **José Manuel Navarro Cuartero**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75896777C, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Manuel Navarro Cuartero

Granada a X de Septiembre de 2026.

D. **José Miguel Mantas Ruiz**, Profesor del Área de XXXX del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado **Estudio e implementación paralela de métodos semiimplícitos multipaso, Resolución de Ecuaciones de Advección-Reacción-Difusión**, ha sido realizado bajo su supervisión por **José Manuel Navarro Cuartero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a X de Septiembre de 2026.

El director:

José Miguel Mantas Ruiz

Agradecimientos

Poner aquí agradecimientos...

Índice

Introducción	1
Conceptos previos	3
Métodos numéricos y EDOs	3
Tipos de métodos numéricos	4
Método de Euler	5
Paralelismo	7
Métodos de Resolución	10
Runge-Kutta	10
Adams-Bashford	12
Adams-Bashford-Moulton	14
Problemas	16
Implementación	17
Secuencial	17
OpenMP	19
Paralelismo por Operaciones	20
Paralelismo por Componentes	21
Mediciones	22
CUDA	26
Mediciones	26

Introducción

Los métodos numéricos son un conjunto de técnicas y algoritmos matemáticos diseñados para encontrar soluciones aproximadas a problemas que, en muchos casos, no pueden resolverse de manera exacta mediante el análisis o su resolución sería muy costosa. Se utilizan asiduamente en diversos campos de la ciencia, ingeniería, economía y muchas otras disciplinas para resolver ecuaciones diferenciales, integrales, sistemas de ecuaciones, aproximación e interpolación, integración numérica entre otros. Es en el primero de estos casos en el que nos centraremos. Si bien es cierto que existen diversas técnicas para la resolución de ecuaciones diferenciales ordinarias y de sistemas de ecuaciones diferenciales, éstas solo se pueden aplicar cuando dichas ecuaciones se ajustan a un patrón concreto.

En el mundo de los métodos numéricos hay infinitud de formas de resolver las EDOs, y tantas o más aplicaciones de las mismas a problemas del mundo real. Escoger el método correcto tiene que ver tanto con la exactitud de la solución, como con su coste, de potencia de computación y tiempo.

Es en este último punto en el que nos centraremos en los siguientes capítulos, ya que si bien para problemas pequeños, algo tan simple como el método de Euler explícito podría bastarnos, posteriormente plantearemos problemas mucho más complejos, que requerirán soluciones que estén a la altura. Por tanto, no se trata solamente de hallar una solución, sino de hacerlo rápida y eficientemente.

Para esto, partiremos de 4 problemas cuya implementación secuencial y soluciones esperadas ya conocemos de forma previa, y aplicaremos para su resolución 3 métodos numéricos distintos, que a su vez implementaremos con dos tecnologías para paralelizar distintas, OpenMP y CUDA, es decir, paralelismo a nivel de CPU y GPU.

Nuestro objetivo será implementar de forma fiel y exacta tanto los problemas como los métodos de resolución, para posteriormente poder comparar las ganancias en tiempo de ejecución al pasar de una tecnología a otra.

La idea detrás de todo esto es relativamente simple; un algoritmo que utilice 4 o 16 hebras será mejor que uno secuencial. Y uno que utilice 256 será mejor que todos ellos. Aunque la realidad no siempre va de la mano de nuestras ideas, buscaremos conseguir las mayores ganancias aprovechando al máximo todas las herramientas que ponen a nuestra disposición tanto CUDA

como OpenMP. Haremos por tanto una serie de mediciones y buscaremos probar nuestra hipótesis.

Conceptos previos

Métodos numéricos y EDOs

Una ecuación diferencial es aquella que relaciona una función ($f(x)$ ó y), sus variables (x) y sus derivadas ($f'(x)$, dy/dx ó y'). Para resolver una ecuación diferencial se debe encontrar una función que satisfaga dicha ecuación. Dentro de las ecuaciones diferenciales nos encontramos las ecuaciones diferenciales ordinarias, EDOs, que contienen derivadas respecto a una sola variable independiente. Más adelante nos centraremos en este tipo en particular. En oposición a éstas estarían las ecuaciones diferenciales parciales, en las que la función $f(x)$ depende de más variables independientes. [Molero et al., 2007]

El orden de una ecuación diferencial equivale al orden de la mayor derivada que aparece en la ecuación. Además, será lineal cuando la variable dependiente y todas sus derivadas son de primer grado (no puede haber $f(x)^2$), y los coeficientes de la variable dependiente y sus derivadas dependen de la variable independiente.

A modo de ejemplo simple podríamos considerar una ecuación diferencial ordinaria de primer orden como:

$$\frac{dy}{dt} = -2y, y(0) = 1$$

Donde además de la EDO, se nos está dando su valor inicial. En este caso en particular, sabemos de forma analítica que su solución exacta es $y(t) = e^{-2t}$, y podríamos por tanto calcular cualquier y_n que deseemos, pero si por cualquier razón no pudiéramos resolverla analíticamente deberíamos recurrir a los métodos numéricos. A través de ellos, y partiendo del valor y_n que se nos proporciona, calcularíamos de forma iterativa y_{n+1} hasta llegar al valor n que busquemos.

Podríamos utilizar, por ejemplo, el método de Euler, del que hablaremos más adelante. Antes de ello podemos entrar en algo más de detalle respecto a algunos otros conceptos que iremos usando.

Tipos de métodos numéricos

Podemos dividir a los métodos en tres grupos, explícitos, implícitos y semiimplícitos. La división se hará en función de cómo se consigue el siguiente paso, es decir, de cómo obtenemos y_{n+1} a partir de y_n ; de si aparecen valores futuros o no en la fórmula que lo calcula. Tendremos como expresión general la siguiente: [Molero et al., 2007]

$$y_{n+1} = y_n + \Delta t \cdot \Phi(x_n, y_n, y_{n+1}, \Delta t) \quad (1)$$

En la cual será la función incremento Φ la que dirima a cuál de los grupos pertenece el método con el que estemos trabajando.

Los métodos explícitos son aquellos en los que siguiente término se puede obtener directamente, sólo haciendo cálculos con datos de los que ya se dispone. Dicho de otra forma, son aquellos en los que y_{n+1} no aparece en Φ , sólo aparece en la parte izquierda de la ecuación. Entre ellos nos encontraremos el Euler explícito, Runge-Kutta, o Adams-Bashford, los tres los veremos posteriormente. Los métodos explícitos suelen ser más fáciles de implementar, ya que no requieren resolver ecuaciones, y suelen ser más rápidos. Por otro lado, tienden a ser menos estables, especialmente en problemas rígidos (*stiff*).

Al otro lado del espectro nos encontraríamos a los métodos implícitos, que son aquellos en los que y_{n+1} sí aparece en la parte de la derecha de la igualdad, lo cuál nos obligaría a resolver una ecuación que podrá ser, o no, lineal. Estos métodos son muy estables, y son más adecuados para resolver problemas rígidos (*stiff*). Como parte negativa tienen que requieren resolver ecuaciones en cada paso, y a veces de forma iterativa, lo que los acaba haciendo mucho más costosos computacionalmente. Entre ellos nos encontramos algunos métodos como el Euler implícito, Adams-Moulton, o los métodos BDF.

Como tercer grupo tendríamos los métodos semiimplícitos. Éstos representan una especie de "solución" a los diferentes problemas que traen consigo los métodos explícitos e implícitos. En este grupo, aunque existe una parte implícita, ésta es más simple, ya que o bien la ecuación a resolver es lineal, o bien se separa la parte rígida para tratarla implícitamente y el resto se trata de forma explícita. Esto es útil cuando el problema tiene términos rígidos (*stiff*) como la difusión, que deben tratarse implícitamente para que el método sea estable incluso con pasos de tiempo grandes, y términos suaves o de comportamiento hiperbólico como la advección, que se pueden tratar explícitamente para evitar resolver sistemas grandes o no lineales. Los métodos semiimplícitos representan un compromiso entre la estabilidad de los explícitos y la velocidad de los implícitos. Como hemos mencionado, podremos dividir a la EDO en dos partes:

$$y' = f(x, y) = f_E(x, y) + f_I(x, y) \quad (2)$$

En este ejemplo f_E es la parte explícita, suave (no *stiff*), y f_I es la parte que queremos tratar implícitamente. Algunos métodos semiimplícitos son los métodos IMEX (*implicit-explicit*), los métodos linealmente implícitos Runge-Kutta (*LIRK*), [Hairer et al., 1993] o el método Adams-Bashford-Moulton (AB-AM predictor-corrector). Es con este último con el que trabajaremos.

Otra categorización con la que trabajaremos serán los métodos de un solo paso, y los métodos multipaso. En los métodos de un solo paso, para obtener el siguiente valor con el que se tratará, el siguiente paso, sólo se utiliza información relativa al paso anterior. Es decir, como podemos ver en 1, para obtener y_{n+1} sólo se utilizaría y_n .

Sin embargo, los métodos multipaso usan varios valores anteriores de la solución para calcular el siguiente paso en el tiempo, en lugar de basarse solo en el valor actual. Es decir, que para obtener y_{n+1} se podrían tomar pasos intermedios que posteriormente se desechen (como en Runge-Kutta), o utilizar pasos más antiguos como y_{n-1} o y_{n-2} . Para problemas de gran tamaño, los métodos multipaso permiten alcanzar resultados con gran precisión y de una forma más eficiente que los métodos de un paso como el de Euler o el de Runge-Kutta.

Método de Euler

Como hemos comentado, el método de Euler es uno de los métodos numéricos más simples de los que disponemos para la resolución de EDOs. Se trata de un método explícito de un solo paso que se construye a partir de una sencilla idea geométrica: aproximar el valor de la solución en cada nodo por el valor que proporciona la recta tangente a la solución trazada desde el punto correspondiente al nodo anterior. [Gallardo, 2018]

Emplearíamos la siguiente fórmula:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) \quad (3)$$

Aplicando la misma al ejemplo de valor inicial que mencionamos antes, $\frac{dy}{dt} = -2y$, $y(0) = 1$, sabemos que y_n corresponde a la aproximación numérica de $y(t_n)$, y Δt representa la variación en el tiempo paso a paso que estemos considerando. Además, en nuestro caso, $f(t_n, y_n)$ sería $-2 \cdot y_n$, y se nos da el valor inicial $t_0 = 0.0$. Por tanto, tomando estos valores, con un $\Delta t = 0.1$, y $t_f = 1.0$, calcularemos los sucesivos valores para $t = 0.1, 0.2, \dots, 1.0$.

El primer paso sería por tanto, partiendo de los valores iniciales, calcular el $t = 0.1$, a saber, $y_1 = y_0 + \Delta t \cdot f(t_0, y_0) = 1 + 0.1 \cdot (-2 \cdot 1) = 0.8$. Si esto lo repitiéramos de forma sucesiva obtendríamos los siguientes resultados, los

cuales además compararemos con el valor real $y(t) = e^{-2t}$ para obtener el error absoluto ε_a y el error relativo ε_r . Definiremos al error absoluto como la diferencia entre el valor real y nuestra aproximación, y al error relativo como el cociente entre el error absoluto y el valor real, en porcentaje.

Paso n	t_n	$y_n(\text{Euler})$	$y_n(\text{Exacta})$	ε_a	ε_r
0	0.0	Valor Inicial = 1.0		-	-
1	0.1	0.8	0.81873	0.01873	2.29
2	0.2	0.64	0.67032	0.03032	4.52
3	0.3	0.512	0.54881	0.03681	6.71
4	0.4	0.4096	0.44933	0.03973	8.84
5	0.5	0.32768	0.36788	0.04020	10.93
6	0.6	0.26214	0.30119	0.03905	12.97
7	0.7	0.20972	0.24660	0.03688	14.96
8	0.8	0.16777	0.20190	0.03412	16.90
9	0.9	0.13422	0.16530	0.03108	18.80
10	1.0	0.10737	0.13534	0.02796	20.66

Tabla 1: Comparativa entre Euler y valor real

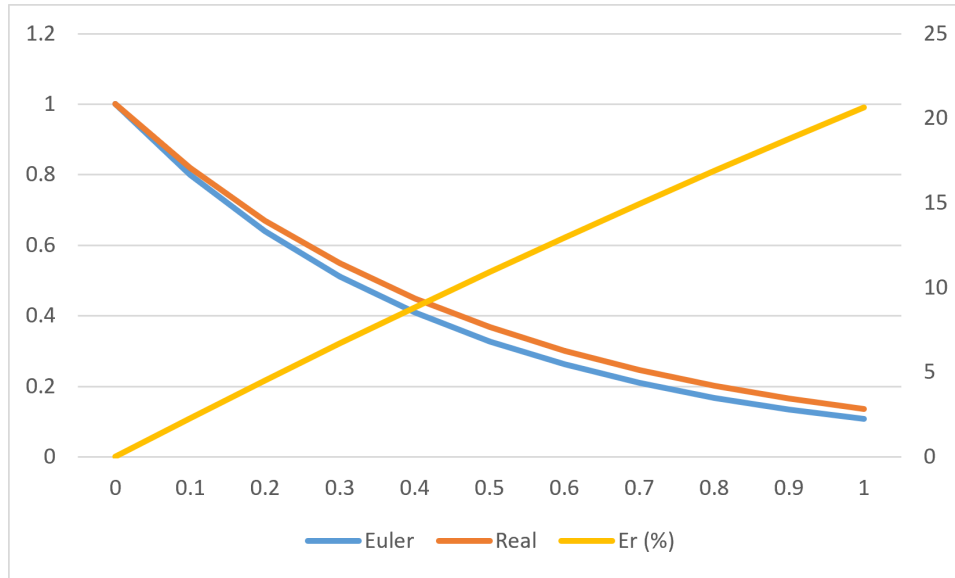


Figura 1: Comparativa entre Euler y valor real

Como podemos observar en la tabla 1 y en la figura 1, los valores que obtenemos con el Euler explícito son próximos a los reales (eje Y izquierdo); el error absoluto se mantiene estable por debajo de 0.05, e incluso disminuye. Sin embargo, el error relativo (eje Y derecho) aumenta de forma lineal con

cada iteración, y para la última alcanza un preocupante 20 %. Esto nos indica que para este problema, quizás el método numérico que hemos utilizado o el Δt que hemos considerado no son los más indicados.

Paralelismo

Las dos tecnologías que vamos a utilizar para introducir el paralelismo en nuestro código y las mejoras que trae consigo son OpenMP y CUDA, es decir, paralelismo a nivel de CPU, y a nivel de GPU, respectivamente.

Por muy rápida que sea nuestra CPU, siempre llegaremos a un punto en que, bien por la complejidad de los problemas, bien por el tamaño que se maneje, una sola CPU con un código secuencial no va a ser suficiente. Es aquí donde entra el paralelismo, y es aquí donde nos centraremos.

La programación paralela es el conjunto de técnicas, modelos y arquitecturas que permiten ejecutar múltiples operaciones de forma simultánea, explotando el hardware moderno (CPU multicore, GPU masivamente paralelas, aceleradores, clústeres). Su objetivo es reducir el tiempo de ejecución, aumentar el rendimiento y permitir resolver problemas que, como hemos comentado, serían inviables en código secuencial.

El paralelismo surge cuando un problema puede descomponerse en sub-tareas independientes que pueden ejecutarse al mismo tiempo. En términos formales, un programa es paralelizable si existe una partición de su espacio de trabajo en unidades que no presentan dependencias de datos que impidan su ejecución concurrente. Esto implica analizar dependencias de datos, localidad y acceso a memoria, granularidad de las tareas, y coste de sincronización.

Si quisiéramos clasificar los distintos modelos de paralelismo nos podríamos encontrar, por un lado con el paralelismo de datos, que consiste en aplicar la misma operación a múltiples elementos de un conjunto, a través de la vectorización, de bucles paralelos, o de operaciones *element-wise*. Dentro de esta categoría nos encontraríamos a OpenMP, CUDA, OpenCL, o AVX, entre otros.

Por otro lado tendríamos el paralelismo de tareas, que consiste en ejecutar tareas heterogéneas en paralelo, como pipelines, árboles de tareas, o grafos de dependencias. Este modelo es usado por frameworks como TBB, Cilk, OpenMP tasks o runtimes de grafos. Por último el paralelismo híbrido combina ambos, buscando el paralelismo de tareas, con paralelismo de datos dentro de cada tarea.

Algunos conceptos muy importantes que nos encontraremos son la concurrencia y el paralelismo. La concurrencia hace referencia a que varias tareas pueden ocurrir a la vez, o que ocurren simultáneamente de forma lógica. Una CPU *single-core* puede cambiar rápidamente de una tarea a otra si no tienen dependencia entre ellas y son concurrentes. Mientras que el paralelismo

trata sobre ejecutar físicamente varias tareas al mismo tiempo, por ejemplo a través de varios núcleos (como en OpenMP) o de varias hebras (como en CUDA).

El speedup será la forma en la que mediremos las mejoras en la ejecución conforme aumenta el paralelismo. En nuestro caso, lo calcularemos dividiendo el tiempo de ejecución del programa secuencial, entre el paralelizado. Como todo código por bien optimizado que esté va a tener una parte secuencial que no pueda paralelizarse, buscaremos que el speedup aumente tanto como sea posible. Por ejemplo, pasando de un código secuencial a uno con 4 hebras OpenMP, el speedup ideal sería de 4, aunque entendemos que no será posible conseguirlo, pero sí acercarse lo suficiente. [Schryen, 2024]

Otro concepto relacionado sería el de la eficiencia, que sería el cociente entre speedup y el número de cores, o de hebras, empleado para obtener ese speedup. La eficiencia estará entre 0 y 1 (o entre 0% y 100%), y medirá cuánto se están aprovechando los recursos paralelos.

Al hilo de esto último, la Ley de Amdahl [Amdahl, 1967] nos indica que, con un tamaño de problema constante y aumentando el número de procesadores, la parte no paralelizable del programa va a limitar nuestro speedup. Podemos ilustrarla tal que:

$$S(p) = \frac{1}{(1 - f) + f/p} \quad (4)$$

Siendo p el número de procesadores, f la fracción paralelizable del programa, $1 - f$ por tanto sería la parte secuencial, y $S(p)$ sería el speedup máximo teórico. La interpretación que podemos hacer es que cuando aumentan los hilos, la parte no paralelizable satura el speedup. Por ejemplo, con un 95% del programa paralelizable, e infinitos procesadores, el speedup nunca superará 20. Trayendo esto a un contexto más realista, lo que se nos indica es que, aunque obviamente aumentar los núcleos o los hilos es efectivo, y reduciremos los tiempos de ejecución, llegará un número a partir del cual ya no merezca la pena.

También debemos mencionar la Ley de Gustafson [Gustafson, 1988], que parte de una premisa distinta, aumentar el tamaño del problema cuando tenemos más procesadores. A saber:

$$S(p) = p - (1 - f) \cdot (p - 1) \quad (5)$$

Aquí la interpretación que hacemos es que al aumentar el tamaño del problema, aumentamos la parte paralelizable, mientras que la secuencial se mantiene estable. Esto juega a nuestro favor, ya que trataremos grandes cantidades de datos. La ley de Gustafson aporta un punto de vista más optimista al binomio procesadores-eficiencia, ya que nos dice que aunque la eficiencia no pueda ser perfecta, a medida que el problema y los hilos crezcan, podrá acercarse mucho.

Otro concepto importante será el de la sincronización. Cuando hilos diferentes deban compartir algún dato entre ellos, o necesiten que haya terminado cierta tarea antes de empezar la siguiente, deberá haber una barrera en la que los hilos se esperen entre ellos. Estas barreras ralentizan mucho la ejecución y se deberá intentar reducirlas tanto como sea posible.

Métodos de Resolución

Para resolver los diferentes problemas hemos implementado tres métodos distintos, a saber: Runge-Kutta, que es explícito y utiliza pasos intermedios ($K1, K2$) para obtener los nuevos pasos; Adams-Bashford, que también es explícito y multipaso, pero utiliza pasos antiguos (como f_{n-1} o f_{n-2}); y Adams-Bashford-Moulton, un predictor-corrector semiimplícito multipaso basado en Adams-Moulton (formalmente implícito) que utiliza Adams-Bashford para predecir y_{n+1} y sustituirlo en $f(t_{n+1}, y_{n+1})$.

Runge-Kutta

Ahora que hemos tratado el método de Euler, podremos dar el siguiente paso hacia Runge-Kutta. Aunque técnicamente ya lo hemos estado viendo, ya que los métodos de Runge-Kutta son generalizaciones de la fórmula básica de Euler. [Segarra, 2020] Por esta razón, se dice que el método de Euler es un método de Runge-Kutta de primer orden.

En cada uno de los órdenes de Runge-Kutta resolveremos una serie de pasos intermedios para hallar el valor final. El de orden 1, como hemos mencionado, coincidiría con Euler, como se puede ver en la ecuación 3.

Para ilustrar este punto, procedemos a pasar a la fórmula de segundo orden, en el que buscaremos encontrar constantes o parámetros w_1, w_2, α, β tal que la fórmula concuerda con un polinomio de Taylor de grado 2. [Zill, 2009]

$$\begin{aligned}y_{n+1} &= y_n + h \cdot (w_1 k_1 + w_2 k_2) \\k_1 &= f(x_n, y_n) \\k_2 &= f(x_n + \alpha h, y_n + \beta h k_1)\end{aligned}\tag{6}$$

Para el uso que haremos en este proyecto, basta decir que esto se puede hacer siempre que las constantes cumplan que:

$$\begin{aligned}w_1 + w_2 &= 1, \quad w_2 \alpha = \frac{1}{2} \text{ y } w_2 \beta = \frac{1}{2} \\w_1 &= 1 - w_2, \quad \alpha = \frac{2}{2w_2} \text{ y } \beta = \frac{1}{2w_2}\end{aligned}$$

Este es un sistema algebraico que posee tres ecuaciones y cuatro incógnitas, lo cual implica un número infinito de soluciones, donde $w_2 \neq 0$, y para nuestra aplicación práctica tomaremos $w_1 = w_2 = \frac{1}{2}$ y $\alpha = \beta = 1$, formando por tanto:

$$\begin{aligned} y_{n+1} &= y_n + h/2 \cdot (k_1 + k_2) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h, y_n + k_1 \cdot h) \end{aligned} \tag{7}$$

De forma análoga, obtendremos el tercer orden:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 4k_2 + k_3) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h, y_n - k_1 \cdot h + 2k_2 \cdot h) \end{aligned} \tag{8}$$

En el método de Runge-Kutta de orden 4 resolveremos diversos pasos que nos servirán para estimar de forma muy aproximada el resultado. Será en éste en el que nos centraremos a la hora de las mediciones. Partiendo de un valor inicial $dy/dt = f(t, y)$, $y(t_0) = y_0$, y de un valor de salto $h > 0$, calcularemos iterativamente los valores tal que:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h/2, y_n + k_2 \cdot h/2) \\ k_4 &= f(t_n + h, y_n + k_3 \cdot h) \end{aligned} \tag{9}$$

Aplicado en un contexto de programación, tendremos una clase *RungeKutta* con una función principal que aplicará el cálculo y algunas auxiliares que se explicarán más adelante. Dicha función principal tendrá el siguiente diseño, en el que se harán diversas llamadas a la evaluación de la derivada de cada problema en cuestión (aquí lo llamaremos *feval*):

Algoritmo 1 Runge-Kutta de Orden 4

```
 $Y_n \leftarrow Y_0$   
 $t_n \leftarrow t_0$   
while  $t_n < t_f$  do  
   $K1 \leftarrow f_{eval}(t_n, Y_n)$   
   $K2 \leftarrow f_{eval}(t_n + h/2, Y_n + K1 \cdot h/2)$   
   $K3 \leftarrow f_{eval}(t_n + h/2, Y_n + K2 \cdot h/2)$   
   $K4 \leftarrow f_{eval}(t_n + h, Y_n + K3 \cdot h)$   
   $Y_n \leftarrow Y_n + h/6 \cdot (K1 + 2 \cdot K2 + 2 \cdot K3 + K4)$   
   $t_n \leftarrow t_n + h$   
end while  
 $Y_f \leftarrow Y_n$ 
```

Como podemos ver, la aplicación del algoritmo es relativamente fácil de entender, se trabaja con 4 vectores auxiliares en los que hacemos los cálculos intermedios, que luego se suman ponderadamente. Como cada evaluación K depende de la anterior, no podremos hacer una operación vectorial en la que se evalúen los cuatro K en paralelo, sino que será en las otras operaciones, las evaluaciones de cada miembro, las sumas, y las multiplicaciones de cada vector, en las que podremos entrar a trabajar el paralelismo para mejorar la eficiencia de nuestro código.

Adams-Bashford

A diferencia de Runge-Kutta, donde sólo considerábamos un valor y_n para obtener y_{n+1} , en los métodos de Adams-Bashford, al ser multipaso, sí tendremos en cuenta más puntos, como y_{n-1} o y_{n-3} . En este proyecto nos hemos centrado en los métodos de paso fijo, es decir, que la diferencia entre y_n y y_{n-1} es constante (e igual a Δt). Si bien existen otros métodos de paso variable, no serán tratados.

De forma general, tenemos que un método lineal de k pasos viene determinado por una expresión tal que: [Molero et al., 2007]

$$\sum_{j=0}^k \alpha_j \cdot z_{n+j} = h \cdot \sum_{j=0}^k \beta_j \cdot f(x_{n+j}, z_{n+j}) , n \geq 0 \quad (10)$$

Para poder aplicar esta fórmula necesitaremos unos valores de arranque, que serán iguales al valor de k . Como en los problemas de valor inicial sólo se proporciona el primero de ellos y_0 , los valores restantes deberemos conseguirlos aplicando otro método (que no sea multipaso), como por ejemplo, Runge-Kutta.

La expresión general de un método de Adams-Bashford de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_{k-1} \cdot \nabla^{k-1}) f_n$$

donde

$$\gamma_i = \frac{1}{i! h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_n) \dots (x - x_{n-i+1}) \cdot dx \quad (11)$$

Aplicando la anterior ecuación 11, obtendríamos los distintos órdenes con los que trabajaremos. Al igual que con Runge-Kutta, el Adams-Bashford de orden 1 coincide con Euler explícito (ecuación 3). De igual manera, obtendríamos Adams-Bashford de 2 pasos:

$$y_{n+1} = y_n + h/2 \cdot (3f_n - f_{n-1}) \quad (12)$$

Este será el primer caso en el que vemos aplicada la teoría de los métodos multipaso, ya que, para obtener el paso y_2 , por ejemplo, deberíamos tener de antemano f_1 y f_0 , para los que a su vez necesitaríamos (a través del arranque con Runge-Kutta) y_1 y y_0 . Una vez hecho el arranque inicial para obtener y_2 , ya sí podríamos operar iterativamente sólo con Adams-Bashford. El de orden 3 sería:

$$y_{n+1} = y_n + h/12 \cdot (23f_n - 16f_{n-1} + 5f_{n-2}) \quad (13)$$

Y por último tendríamos la expresión de orden 4, que será con la que principalmente trabajaremos:

$$y_{n+1} = y_n + h/24 \cdot (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \quad (14)$$

Llevándonos este método a la programación, tendremos una clase *Adams-Bashford* con una función *aplicar* que se encargará de, primero, realizar un arranque con Runge-Kutta de orden 4 (algoritmo 1) y posteriormente poner en funcionamiento un bucle que ejecute la expresión tantas veces como esté establecido. En función del lenguaje utilizado y del tipo de paralelismo, las operaciones vectoriales funcionarán de una forma u otra, pero el planteamiento será el mismo, con llamadas a la evaluación de cada problema en cuestión (*feval*). El algoritmo para aplicar el método, siendo k el orden del mismo, será:

Algoritmo 2 Adams-Bashford general

```
 $Y_n \leftarrow \text{arranqueRK4}(y_{n-1}) \quad \triangleright 1 \leq n < k$   
 $F_n \leftarrow \text{feval}(Y_n) \quad \triangleright 0 \leq n < k$   
 $t_n \leftarrow t_0 + (k-1) \cdot h$   
while  $t_n < t_f$  do  
     $Y_k \leftarrow \text{AdamsBashford-K}(Y_{k-1}, F_n) \quad \triangleright 0 \leq n < k$   
     $F_k \leftarrow \text{feval}(Y_k)$   
     $\text{swap}(Y_k, Y_{k-1})$   
     $\text{swap}(F_n, F_{n-1}) \quad \triangleright 1 \leq n \leq k$   
     $t_n \leftarrow t_n + h$   
end while  
 $Y_f \leftarrow Y_{k-1}$ 
```

La idea detrás del algoritmo es que, tras el arranque de los $k-1$ vectores y_n , y la evaluación de los k vectores f_n , donde k es igual al orden de A-B que estemos aplicando, entrar en el bucle iterativo. En él, iteraremos sobre 2 vectores y_n , a saber, y_k y y_{k-1} , y sobre $k+1$ vectores f_n .

Una vez en el bucle, calculamos el nuevo valor de y_k , y con él, f_k . Posteriormente se hará un swap de vectores tal que y_k pasa a ser y_{k-1} , y todos los vectores f_n pasan a ser f_{n-1} . Así tendremos los datos preparados para la siguiente iteración.

Adams-Bashford-Moulton

Los métodos de Adams-Moulton tienen un funcionamiento similar a los de Adams-Bashford, con la salvedad de que estos son implícitos. Esto quiere decir que para calcular y_{n+1} se hace uso de $f(t_{n+1}, y_{n+1})$. Para resolver este problema utilizaremos un sistema Predictor-Corrector dándonos un método semiimplícito.

La expresión general de un método Adams-Moulton puro de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_k \cdot \nabla^k) f_{n+1}$$

donde

$$\gamma_0 = 0 \text{ y } \gamma_i = \frac{1}{i!h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_{n+1}) \dots (x - x_{n-i+2}) \cdot dx \quad (15)$$

Y por tanto, las ecuaciones correspondientes a los mismos son las siguientes: [Segarra, 2020]

Adams-Moulton orden 0 (AM1)

$$y_n = y_{n-1} + h \cdot f_n \quad (16)$$

Adams-Moulton orden 1 (AM2)

$$y_{n+1} = y_n + h/2 \cdot (f_{n+1} + f_n) \quad (17)$$

Adams-Moulton orden 2 (AM3)

$$y_{n+1} = y_n + h/12 \cdot (5f_{n+1} + 8f_n - f_{n-1}) \quad (18)$$

Adams-Moulton orden 3 (AM4)

$$y_{n+1} = y_n + h/24 \cdot (9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \quad (19)$$

Adams-Moulton orden 4 (AM5)

$$y_{n+1} = y_n + h/720 \cdot (251f_{n+1} + 646f_n - 264f_{n-1} + 106f_{n-2} + 19f_{n-3}) \quad (20)$$

Será en el AM5 en el que nos centraremos principalmente.

Es intuitivamente obvio que llegados a este punto no podríamos resolver estos métodos con las herramientas de las que disponemos. ¿Cómo llegamos a f_{n+1} sin disponer todavía de y_{n+1} ? Para resolver este problema, empleamos un sistema Predictor-Corrector mediante el cual utilizamos Adams-Bashford para hacer una primera aproximación de f_{n+1} , y la evaluamos para posteriormente hacer una segunda aproximación usando Adams-Moulton. El esquema será el siguiente, donde k es igual al orden:

Algoritmo 3 Adams-Bashford-Moulton general

$Y_n \leftarrow \text{arranqueRK4}(Y_{n-1})$	$\triangleright 1 \leq n < k$
$F_n \leftarrow \text{feval}(Y_n)$	$\triangleright 0 \leq n < k$
$t_n \leftarrow t_0 + (k-1) \cdot h$	
while $t_n < t_f$ do	
$Y_{pred} \leftarrow \text{AdamsBashford-K}(Y_{k-1}, F_{k-1})$	$\triangleright$ Aplicar orden k
$F_{pred} \leftarrow \text{feval}(t_{n+h}, Y_{pred})$	
$Y_{corr} \leftarrow \text{AdamsMoulton-K}(Y_{k-1}, F_{pred})$	$\triangleright$ Aplicar orden k
$F_{corr} \leftarrow \text{feval}(t_{n+h}, Y_{corr})$	
$Y_k \leftarrow \text{AdamsMoulton-K}(Y_{k-1}, F_{corr})$	$\triangleright$ Aplicar orden k
$F_k \leftarrow \text{feval}(t_{n+h}, Y_k)$	
$\text{swap}(y_k, y_{k-1})$	
$\text{swap}(f_n, f_{n-1})$	$\triangleright 1 \leq n \leq k$
$t_n \leftarrow t_n + h$	
end while	
$Y_f \leftarrow Y_{k-1}$	

El funcionamiento del algoritmo acaba siendo muy similar al esquema de Adams-Bashford, en tanto que hacemos el arranque con Runge-Kutta y posteriormente iteramos sobre los vectores Y_n y F_n . En este caso, añadimos además los vectores predictor y corrector, Y_{pred} y F_{corr} , que nos permiten aplicar la idea central de este método, manteniendo una buena eficiencia pero con resultados más precisos.

Problemas

Para poner en funcionamiento la implementación de estos métodos y de la paralelización de la que hablaremos a continuación, hemos tomado cuatro problemas distintos que nos permitan probar extensivamente ambas tecnologías, tanto OpenMP como CUDA. Los problemas en cuestión son:

SimpleAdvDiff y AdvDiff1D son problemas unidimensionales de advección - difusión en los que se trabajará con vectores que tendrán un número de EDOs igual al tamaño del vector y estarán formados por dos partes, una rígida (*stiff*) y una no rígida. Si bien servirán para establecer las bases del proyecto, nos centraremos más en los otros dos problemas.

Brusselator1D, el modelo brusselator es un tipo de modelo de reacción-difusión que fue desarrollado por la Brussels School. Este sistema está diseñado para analizar el comportamiento de sistemas químicos que presentan oscilación no lineal. [Hunsdorfer and Verwer, 2003] Este problema tendrá 2 componentes, y por tanto el número de EDOs será el doble del tamaño que se elija.

Brusselator2D, este modelo es similar a su versión unidimensional, pero incorpora una componente extra reactiva. [Mara'beh et al., 2024] Además, está discretizado uniformemente en una malla de $N \times N$ puntos, por lo tanto, su número de EDOs será el doble del cuadrado del tamaño de problema.

Implementación

Secuencial

Comenzaremos a partir de la implementación secuencial de los problemas. [Mantas, 2024]

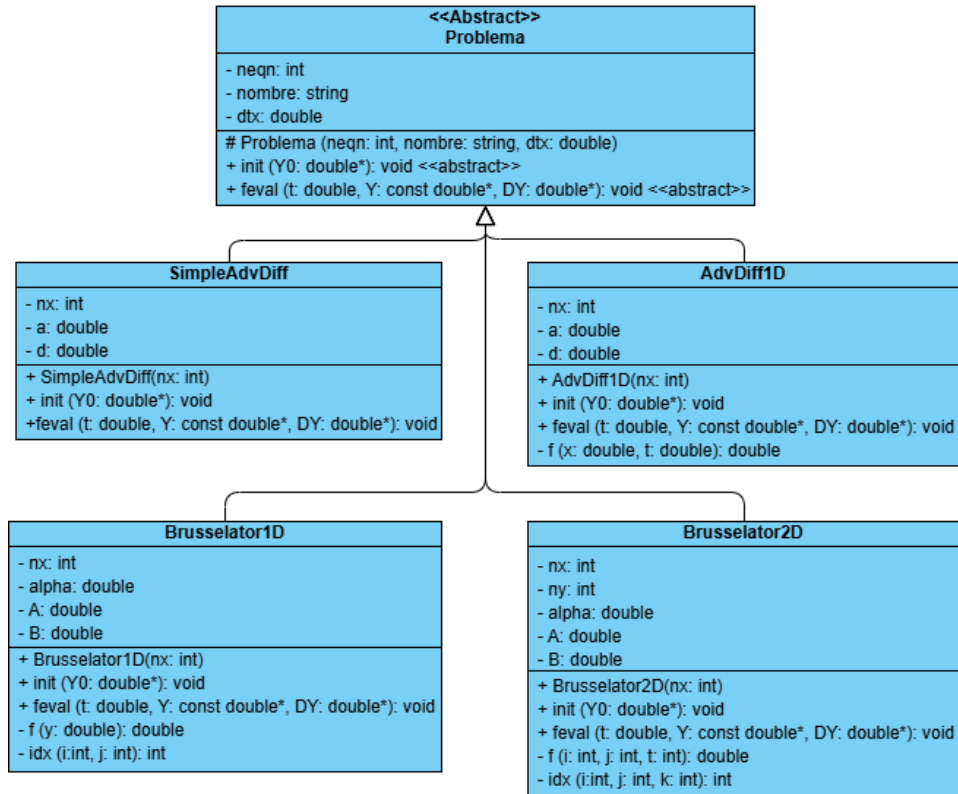


Figura 2: Diagrama de Clases de los problemas

Como comentamos anteriormente, no nos centraremos en la resolución de los problemas *per se*, puesto que ésta se nos da ya hecha y corregida, sino que nuestros esfuerzos irán en implementar los métodos de resolución en ambas formas de paralelismo, y buscar la máxima eficiencia. Para facilitar

la implementación y la cohesión del código, definimos una clase abstracta *Problema* que aúne todo el comportamiento común a los cuatro problemas.

El siguiente paso, por tanto, será realizar una implementación secuencial de los métodos, partiendo de los algoritmos que previamente hemos comentado, Runge-Kutta, Adams-Bashford, y Adamd-Bashford-Moulton, a saber, algoritmos 1, 2, y 3, respectivamente. El lenguaje elegido será C++, para luego poder añadir la parte de OpenMP o CUDA.

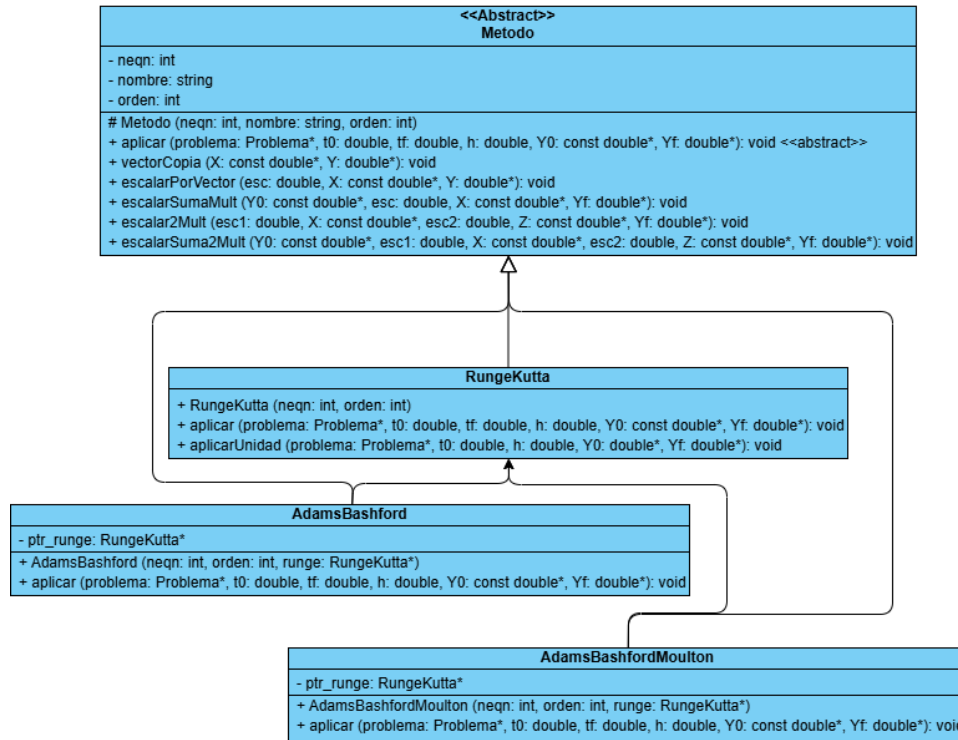


Figura 3: Diagrama de Clases de los métodos

El planteamiento que se hace con los métodos es similar al de los problemas, una clase abstracta padre de la cual puedan heredar los métodos. Además, en la clase padre *Metodo* hemos implementado diversas funciones auxiliares que nos van a facilitar el trabajar con vectores. Con este diseño, a través de la herencia y el polimorfismo, podríamos aumentar el alcance de nuestro proyecto añadiendo nuevos métodos o más problemas sin dificultad ninguna.

Ahora que tenemos una primera maqueta funcional (aunque poco eficiente), podremos comenzar con el trabajo en paralelo.

OpenMP

OpenMP es un modelo de programación paralela para arquitecturas de memoria compartida, diseñado para proporcionar una interfaz portable, escalable y de alto nivel que permita explotar el paralelismo en CPUs multinúcleo sin recurrir a APIs de hilos de bajo nivel. Su diseño se basa en directivas de compilador, rutinas de biblioteca y variables de entorno, y sigue el modelo de ejecución *fork-join*, donde un hilo maestro crea y sincroniza equipos de hilos que ejecutan regiones paralelas. [Jin and Milfeld, 1997]

OpenMP surge en 1997, cuando Intel impulsa la creación del OpenMP Architecture Review Board (ARB), un consorcio formado por fabricantes de hardware y software (Intel, IBM, AMD, HP, NEC, Cray, Fujitsu, Nvidia, Oracle, entre otros) con el objetivo de definir un estándar común para paralelismo en memoria compartida. La última versión de OpenMP, la 6.0, data de 2024.

OpenMP implementa paralelismo mediante *multithreading*. El funcionamiento es relativamente simple, el programa comienza con un comportamiento secuencial y un hilo maestro. Cuando este hilo se encuentra una región paralela con `#pragma omp parallel`, se crea un equipo de hilos [Arb, 2024]. Los hilos ejecutan el bloque paralelo de forma concurrente, y al finalizar, los hilos se unen y el control vuelve al maestro.

OpenMP se compone de varios elementos principales, el primero de los cuales sería las directivas de compilador, como `#pragma omp parallel`, `#pragma omp for`, o `#pragma omp single`, y sus cláusulas como *schedule*, *shared*, o *default*. Otros elementos importantes son las funciones de biblioteca, como `omp_set_num_threads()` o `omp_get_wtime()`, y las variables de entorno, como `OMP_NUM_THREADS`, `OMP_SCHEDULE`, o `OMP_DYNAMIC`. [Arb, 2024]

Como hemos comentado, OpenMP opera sobre un modelo de memoria compartida en la que todos los hilos ven el mismo espacio de direcciones, en el que las variables pueden ser visibles por todos los hilos (*shared*), o cada hilo tiene su propia copia (*private*). Este modelo evita la necesidad de comunicación explícita entre procesos (como en MPI), simplificando el desarrollo. [Jin and Milfeld, 1997]

De cara a nuestro proyecto, lo primero que debemos tener en cuenta es que no podemos simplemente paralelizar el bucle principal con el que trabajamos (e.g. el bucle `while` en el algoritmo 2) porque cada iteración necesita de la anterior para calcular el siguiente paso. Debemos buscar por tanto otras formas de atacar el problema. En el caso de OpenMP hemos planteado dos formas, paralelismo por operaciones, y por elementos.

Paralelismo por Operaciones

En esta estrategia buscamos que cada función paralelice su propio trabajo. Eso quiere decir que cada operación vectorial, como el *feval()*, o *escalarPorVector*, tiene su propio *#pragma omp parallel*, crea un equipo de hilos, reparte el trabajo, y lo sincroniza.

De forma simplificada, veamos un ejemplo de Adams-Bashford 4:

Algoritmo 4 Implementación de AB4 con Paralelismo por Operaciones

```

 $Y_{n0} \leftarrow Y_0$ 
 $Y_n \leftarrow \text{arranqueRK4}(Y_{n-1}) \quad \triangleright n = \{1, 2, 3\}$ 
 $F_n \leftarrow \text{feval}(t_n, Y_n) \quad \triangleright n = \{0, 1, 2, 3\}$ 
for  $t_n = t_0 + 3h; t_n < t_f; t_n += h$  do
    escalar2Mult(37.0,  $F_{n1}$ , -9.0,  $F_{n0}$ ,  $Y_{aux}$ )
    escalarSuma2Mult( $Y_{aux}$ , 55.0,  $F_{n3}$ , -59.0,  $F_{n2}$ ,  $Y_{aux}$ )
    escalarSumaMult( $Y_{n3}$ ,  $h/24.0$ ,  $Y_{aux}$ ,  $Y_{n4}$ )
     $F_{n4} \leftarrow \text{feval}(t_n + h, Y_{n4})$ 
    swap( $Y_{n4}$ ,  $Y_{n3}$ )
    swap( $F_n$ ,  $F_{n-1}$ )  $\triangleright n = \{1, 2, 3, 4\}$ 
end for
 $Y_f \leftarrow Y_{n3}$ 

```

Como podemos ver, para hacer el cálculo propio de Adams-Bashford de orden 4 (ecuación 14) la hemos dividido en 3 partes utilizando las funciones auxiliares de la clase abstracta *Metodo* de las que ya disponíamos. Dentro de cada una de ellas se creará el grupo de hilos, se repartirá el trabajo (cada hilo tendrá un "trozo" de vector sobre el que trabajar), y una vez finalicen se sincronizará para retornar a la función principal. Por ejemplo, *vectorCopia()*:

Algoritmo 5 Implementación de Función Auxiliar

```

#pragma omp parallel for schedule(static)
for  $i=0; i < neqn; ++i$  do
     $Y[i] \leftarrow X[i]$ 
end for

```

Esta estrategia trae consigo ventajas como la modularidad, ya que cada operación vectorial es independiente y autocontenida, lo cual facilita la depuración y la legibilidad del código.

El principal punto negativo de esta estrategia es que trae consigo un *overhead* alto, ya que cada función tiene una barrera implícita al acabar un *parallel for*, en cada iteración del bucle hay varias llamadas a funciones, y el bucle itera un elevado número de veces. Esto equivale a más paradas de las que serían deseadas, además de que los hilos entran y salen de regiones paralelas constantemente.

Paralelismo por Componentes

Frente al paralelismo de operaciones, tenemos el paralelismo por componentes (o por elementos), en el que sólo tenemos una gran región *parallel* dentro de la cual ocurre todo el paralelismo, y cada hebra calcula su parte de los vectores.

Esto quiere decir que se creará un único equipo de hilos al entrar en el *parallel*, y todos los hilos permanecerán activos e iterarán en el bucle, paso a paso. Dentro del bucle iterativo, habrá diversos *#pragma omp for* que repartirán los índices entre los hilos. Esto además nos permitirá reducir las barreras de sincronización. Para poder comparar con la anterior implementación 4, describimos el mismo algoritmo (14) pero con esta estrategia:

Algoritmo 6 Implementación de AB4 con Paralelismo por Componentes

```

 $Y_{n0} \leftarrow Y_0$ 
 $Y_n \leftarrow \text{arranqueRK4}(Y_{n-1})$   $\triangleright n = \{1, 2, 3\}$ 
#pragma omp parallel for schedule(static)
for  $i=0; i < neqn; ++i$  do
     $F_n[i] \leftarrow \text{feval}_i(t_n, Y_n, i)$   $\triangleright n = \{0, 1, 2, 3\}$ 
end for
#pragma omp parallel
for  $t_n=t_0+3h; t_n < t_f; t_n+=h$  do
    #pragma omp for schedule(static)
    for  $i=0; i < neqn; ++i$  do
         $Y_4[i] = Y_3[i] + h/24 * (55F_3[i] - 59F_2[i] + 37F_1[i] - 9F_0[i])$ 
    end for
    #pragma omp for schedule(static)
    for  $i=0; i < neqn; ++i$  do
         $F_4[i] \leftarrow \text{feval}_i(t_n + h, Y_4, i)$ 
    end for
    #pragma omp single
     $\text{swap}(Y_{n4}, Y_{n3})$ 
     $\text{swap}(F_n, F_{n-1})$   $\triangleright n = \{1, 2, 3, 4\}$ 
end for
 $Y_f \leftarrow Y_{n3}$ 

```

La idea principal detrás de esta estrategia es reducir el *overhead* provocado por tener demasiadas barreras de sincronización, y mejorar el uso de la caché. Para mejorar la implementación hacemos uso de directivas como *schedule (static)*, con la que indicamos cómo deben repartirse las iteraciones entre los hilos. En este caso, se divide el rango de iteraciones en bloques contiguos y de tamaño fijo, y se asigna cada bloque a un hilo antes de empezar la ejecución del bucle. Esto es práctico cuando sabemos de antemano que todas las iteraciones van a costar lo mismo, y no va haber ramas pesadas o

irregulares.

Algunas de las desventajas que tendremos es que el código es más complejo, pues no se puede reutilizar el *feval()* secuencial (que sí lo aprovechábamos en el paralelismo por operaciones), sino que hay que hacer una versión específica para los componentes. De igual manera, no se pueden utilizar las funciones auxiliares, porque están diseñadas para tratar los vectores completos, no elementos individuales.

Mediciones

Ante la pregunta de qué estrategia es mejor, la respuesta sería, como casi siempre en informática, que depende.

En un primer planteamiento, parece que el paralelismo por componentes debería ser mejor. Reduce la creación y destrucción repetitiva de las hebras, y reduce las barreras de sincronización.

Un matiz importante que debemos hacer es que, aunque hablemos de creación y destrucción de hilos iterativa, no siempre ha de ocurrir así. Es cierto que, de forma lógica, el modelo *fork-join* de OpenMP describe de esa forma el comportamiento de los hilos; se crean, se usan, se destruyen. [Jin and Milfeld, 1997] Sin embargo, esto no siempre ocurre así (ya que sería muy costoso).

Existe un concepto llamado *thread pooling*, que describe que el entorno de ejecución podrá devolver los hilos a la *pool* en lugar de destruirlos, y así ahorrarse crearlos posteriormente. [Barlas, 2023] Esto eliminaría la principal desventaja del paralelismo por operaciones.

De forma análoga, el *runtime* no siempre mantiene los hilos activos (como pudiera parecer), aunque no haya finalizado el bloque *parallel*, ya que si el trabajo dentro de cada bloque *omp for* es pequeño, el compilador puede decidir dormir los hilos para ahorrar energía. Esto puede provocar pérdida de tiempo al tener que despertar los hilos, y latencias de sincronización más altas.

Otro dato a tener en cuenta, es que en el planteamiento de nuestros problemas, las llamadas a *feval_i()* son muy baratas, y el trabajo por iteración es pequeño, mientras que *feval()* es más pesado, y en cierta forma se amortiza el *overhead* al hacerse una única llamada en lugar de *neqn* llamadas.

Podemos comenzar, por ejemplo, comparando los tiempos al resolver el problema *Brusselator1D* con 100.000 ecuaciones.

Solver	Hebras	Estrategia	T (s)
Runge-Kutta	1	OMP Componentes	2501.84
		OMP Operaciones	1544.87
	4	OMP Componentes	670.504
		OMP Operaciones	371.93
	16	OMP Componentes	197.065
		OMP Operaciones	180.327
Adams-Bashford	1	OMP Componentes	707.634
		OMP Operaciones	556.051
	4	OMP Componentes	179.559
		OMP Operaciones	130.993
	16	OMP Componentes	56.5174
		OMP Operaciones	66.1485
Adams-Bashford-Moulton	1	OMP Componentes	2914.04
		OMP Operaciones	1772.73
	4	OMP Componentes	785.555
		OMP Operaciones	438.018
	16	OMP Componentes	237.236
		OMP Operaciones	219.918

Tabla 2: Tiempos de Brusselator1D con 100000 ecuaciones

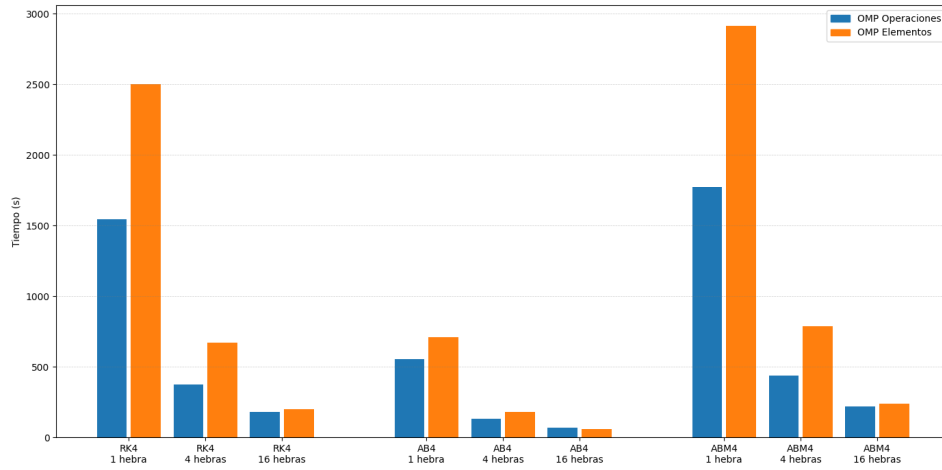


Figura 4: Brusselator1D 100.000 ecuaciones

Como podemos ver, hay una mejora sustancial del tiempo al pasar de 1 a 4 y a 16 hebras, con ambas estrategias de paralelismo, y con los 3 métodos numéricos. Además, se aprecia que la resolución con Adams-Bashford es más rápida que Runge-Kutta, ya que es más directa y no requiere de los

pasos intermedios (K1, K2, K3, K4), y también es mucho más veloz que el predictor-corrector Adams-Bashford-Moulton, ya que éste último llama a Adams-Bashford dentro de su bucle de ejecución. Además, podemos ver que para este tipo y tamaño de problema, el paralelismo por operaciones es más eficiente.

Podemos probar por ejemplo, con el problema más complejo (*Brusselator2D*), y el método que más tarda (*ABM4*), y podemos comparar la evolución del tiempo conforme aumenta el tamaño del problema, a saber, con un número de ecuaciones igual a 45000, 80000 y 125000.

neqn	Hebras	Estrategia	T (s)	Speedup	Eficiencia
45000	1	Operaciones	976.061	1	100
		Componentes	2151.79		
	4	Operaciones	263.081	3.710115896	92.7528974
		Componentes	569.693	3.777104511	94.4276128
	16	Operaciones	180.662	5.402691213	33.7668201
		Componentes	206.744	10.40799249	65.0499531
80000	1	Operaciones	1825.86	1	100
		Componentes	3970.37		
	4	Operaciones	466.282	3.915784868	97.8946217
		Componentes	1049.35	3.78364702	94.5911755
	16	Operaciones	214.67	8.505426934	53.1589183
		Componentes	331.809	11.96582974	74.7864359
125000	1	Operaciones	2692.79	1	100
		Componentes	5982.94		
	4	Operaciones	675.123	3.988591708	99.7147927
		Componentes	1593.61	3.754331361	93.8582840
	16	Operaciones	264.402	10.18445398	63.6528373
		Componentes	445.081	13.44236218	84.0147636

Tabla 3: Brusselator2D con ABM4

Más allá de lo obvio, que conforme aumenta el tamaño del problema, aumentan los tiempos, es cuanto menos destacable la variación en los speedups que se alcanzan en función del aumento de las hebras, en una estrategia y otra.

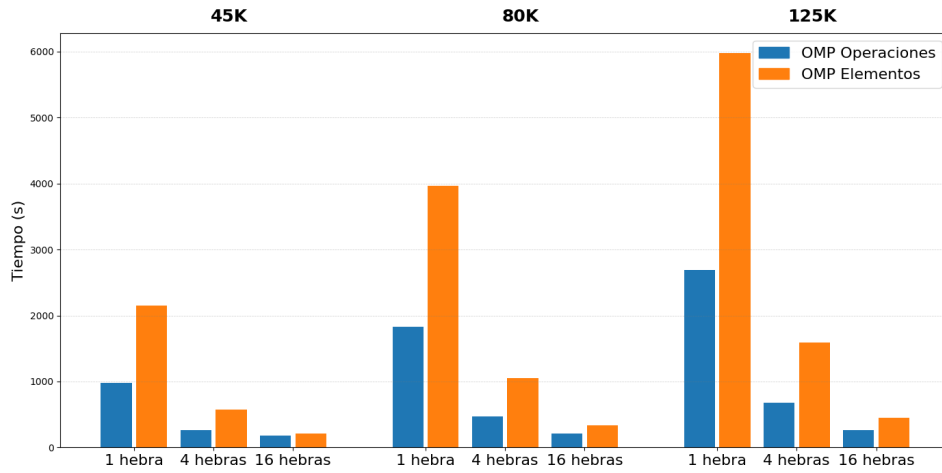


Figura 5: ABM4 resolviendo Brusselator2D

Partiendo de los mismos datos, pero tomando ahora solo el speedup y la eficiencia, y filtrando por la estrategia de las operaciones (que es la más eficiente), tendríamos estas gráficas:

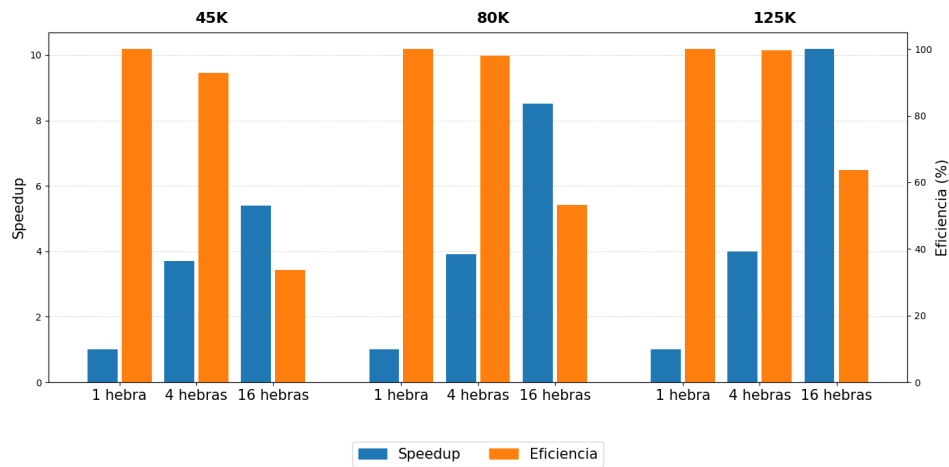


Figura 6: Speedup y Eficiencia

Tenemos en azul y aliadas con el eje Y izquierdo las barras del speedup, y en naranja y con el eje Y derecho, la eficiencia en valores porcentuales. Obviamente, para 1 hebra siempre tendremos un speedup de 1, y una eficiencia de 100 %.

Más allá de eso, podemos ver varias ideas en juego a la vez. Si nos centramos en cualquiera de los 3 tamaños, podemos ver la Ley de Amdahl [Amdahl, 1967] en funcionamiento; con un mismo tamaño de problema, conforme au-

menta el número de hilos, disminuye el speedup, ya que siempre queda una parte secuencial que no se puede paralelizar.

En paralelo a esto, si nos fijamos ahora en las columnas de la eficiencia, podemos ver que, si bien la eficiencia disminuye dentro de un mismo grupo de tamaño (algo esperable), conforme aumenta el tamaño de problema, la eficiencia sí aumenta con él, pasando de un 33 %, a un 53 % y un 63 %, cuando observamos los datos relativos a las 16 hebras. Esto coincide con la ley de Gustafson [Gustafson, 1988], ya que al aumentar el tamaño del problema, crece la parte del programa que es paralelizable, y por lo tanto se pueden usar las hebras de forma más eficiente.

CUDA

Mediciones

Bibliografía

- [Amdahl, 1967] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Spring Joint Computer Conference*.
- [Arb, 2024] Arb, O. (2024). Openmp 6.0 api syntax reference guide  directives and constructs.
- [Barlas, 2023] Barlas, G. (2023). *Multicore and GPU Programming*. Morgan Kaufman Publisher, second edition.
- [Gallardo, 2018] Gallardo, J. M. (2018). *Análisis Numérico de Ecuaciones Diferenciales*. Universidad de Málaga.
- [Gustafson, 1988] Gustafson, J. (1988). Gustafson - reevaluating amdahl's law. *Communications of the ACM*, 31.
- [Hairer et al., 1993] Hairer, E., Norsett, S. P., and Wanner, G. (1993). *Solving Ordinary Differential Equations I*. Springer Berlin Heidelberg.
- [Hunsdorfer and Verwer, 2003] Hunsdorfer, W. and Verwer, J. G. (2003). *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*, volume 33. Springer-Verlag.
- [Jin and Milfeld, 1997] Jin, H. and Milfeld, K. (1997). Openmp application programming interface examples. Technical report, OpenMP Architecture Review Board.
- [Mantas, 2024] Mantas, J. (2024). Vssbdf: Variable stepsize semi-implicit backward differentiation formula solvers in c++.
- [Mara'beh et al., 2024] Mara'beh, R. A., Mantas, J. M., González, P., and Spiteri, R. J. (2024). Performance comparison of variable-stepsize imex sbdf methods on advection-diffusion-reaction models. *Computers and Mathematics with Applications*.
- [Molero et al., 2007] Molero, M., Alcaide, A., Menarguez, T., and Luis, S. (2007). *Análisis Matemático para Ingeniería*. Pearson Educación.

- [Schryen, 2024] Schryen, G. (2024). Speedup and efficiency of computational parallelization: A unifying approach and asymptotic analysis. *Journal of Parallel and Distributed Computing*, 187.
- [Segarra, 2020] Segarra, J. (2020). Runge-kutta y adams-bashford en mathematica. *Revista Ingeniería, Matemáticas y Ciencias de la Información*, 7:13–32.
- [Zill, 2009] Zill, D. G. (2009). *Ecuaciones diferenciales con aplicaciones de modelado*. Brooks/Cole, Cengage Learning.

Listado de Tablas

1.	Comparativa entre Euler y valor real	6
2.	Tiempos de Brusselator1D con 100000 ecuaciones	23
3.	Brusselator2D con ABM4	24

Listado de Imágenes

1.	Comparativa entre Euler y valor real	6
2.	Diagrama de Clases de los problemas	17
3.	Diagrama de Clases de los métodos	18
4.	Brusselator1D 100.000 ecuaciones	23
5.	ABM4 resolviendo Brusselator2D	25
6.	Speedup y Eficiencia	25

Listado de Algoritmos

1.	Runge-Kutta de Orden 4	12
2.	Adams-Bashford general	14
3.	Adams-Bashford-Moulton general	15
4.	Implementación de AB4 con Paralelismo por Operaciones . .	20
5.	Implementación de Función Auxiliar	20
6.	Implementación de AB4 con Paralelismo por Componentes .	21